



Functional-Dependency Theory



Functional-Dependency Theory Roadmap

- We now consider the formal theory that tells us which functional dependencies are implied logically by a given set of functional dependencies.
- We then develop algorithms to generate lossless decompositions into BCNF and 3NF
- We then develop algorithms to test if a decomposition is dependency-preserving



Closure of a Set of Functional Dependencies

- Given a set F set of functional dependencies, there are certain other functional dependencies that are logically implied by F .
 - If $A \rightarrow B$ and $B \rightarrow C$, then we can infer that $A \rightarrow C$
 - etc.
- The set of **all** functional dependencies logically implied by F is the **closure** of F .
- We denote the *closure* of F by F^+ .



Closure of a Set of Functional Dependencies

- We can compute F^+ , the closure of F , by repeatedly applying **Armstrong's Axioms**:
 - **Reflexive rule**: if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$
 - **Augmentation rule**: if $\alpha \rightarrow \beta$, then $\gamma \alpha \rightarrow \gamma \beta$
 - **Transitivity rule**: if $\alpha \rightarrow \beta$, and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$
- These rules are
 - **Sound** -- generate only functional dependencies that actually hold, and
 - **Complete** -- generate all functional dependencies that hold.



Example of F^+

- $R = (A, B, C, G, H, I)$
 $F = \{ A \rightarrow B$
 $A \rightarrow C$
 $CG \rightarrow H$
 $CG \rightarrow I$
 $B \rightarrow H \}$
- Some members of F^+
 - $A \rightarrow H$
 - by transitivity from $A \rightarrow B$ and $B \rightarrow H$
 - $AG \rightarrow I$
 - by augmenting $A \rightarrow C$ with G , to get $AG \rightarrow CG$
and then transitivity with $CG \rightarrow I$
 - $CG \rightarrow HI$
 - by augmenting $CG \rightarrow I$ to infer $CG \rightarrow CGI$,
and augmenting of $CG \rightarrow H$ to infer $CGI \rightarrow HI$,
and then transitivity



Procedure for Computing F^+

- To compute the closure of a set of functional dependencies F :

$F^+ = F$

repeat

for each functional dependency f in F^+

 apply reflexivity and augmentation rules on f

 add the resulting functional dependencies to F^+

for each pair of functional dependencies f_1 and f_2 in F^+

if f_1 and f_2 can be combined using transitivity

then add the resulting functional dependency to F

+

until F^+ does not change any further

- **NOTE:** We shall see an alternative procedure for this task later



Closure of Functional Dependencies (Cont.)

- Additional rules:
 - **Union rule:** If $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta \gamma$ holds.
 - **Decomposition rule:** If $\alpha \rightarrow \beta \gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds.
 - **Pseudotransitivity rule:** If $\alpha \rightarrow \beta$ holds and $\gamma \beta \rightarrow \delta$ holds, then $\alpha \gamma \rightarrow \delta$ holds.
- The above rules can be inferred from Armstrong's axioms.



Closure of Attribute Sets

- Given a set of attributes α , define the **closure** of α **under** F (denoted by α^+) as the set of attributes that are functionally determined by α under F
- Algorithm to compute α^+ , the closure of α under F

result := α ;

while (changes to *result*) **do**

for each $\beta \rightarrow \gamma$ **in** F **do**

begin

if $\beta \subseteq \textit{result}$ **then** *result* := *result* $\cup \gamma$

end



Example of Attribute Set Closure

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B$
 $A \rightarrow C$
 $CG \rightarrow H$
 $CG \rightarrow I$
 $B \rightarrow H\}$
- $(AG)^+$
 1. $result = AG$
 2. $result = ABCG$ ($A \rightarrow C$ and $A \rightarrow B$)
 3. $result = ABCGH$ ($CG \rightarrow H$ and $CG \subseteq AGBC$)
 4. $result = ABCGHI$ ($CG \rightarrow I$ and $CG \subseteq AGBCH$)
- Is AG a candidate key?
 1. Is AG a super key?
 1. Does $AG \rightarrow R$? $==$ Is $R \subseteq (AG)^+$
 2. Is any subset of AG a superkey?
 1. Does $A \rightarrow R$? $==$ Is $R \subseteq (A)^+$
 2. Does $G \rightarrow R$? $==$ Is $R \subseteq (G)^+$
 3. In general: check for each subset of size $n-1$



Uses of Attribute Closure

There are several uses of the attribute closure algorithm:

- **Testing for superkey:**
 - To test if α is a superkey, we compute α^+ , and check if α^+ contains all attributes of R .
- **Testing functional dependencies**
 - To check if a functional dependency $\alpha \rightarrow \beta$ holds (or, in other words, is in F^+), just check if $\beta \subseteq \alpha^+$.
 - That is, we compute α^+ by using attribute closure, and then check if it contains β .
 - Is a simple and cheap test, and very useful
- **Computing closure of F**
 - For each $\gamma \subseteq R$, we find the closure γ^+ , and for each $S \subseteq \gamma^+$, we output a functional dependency $\gamma \rightarrow S$.



Canonical Cover

- Suppose that we have a set of functional dependencies F on a relation schema. Whenever a user performs an update on the relation, the database system must ensure that the update does not violate any functional dependencies; that is, all the functional dependencies in F are satisfied in the new database state.
- If an update violates any functional dependencies in the set F , the system must roll back the update.
- We can reduce the effort spent in checking for violations by **testing a simplified set of functional dependencies that has the same closure as the given set.**
- This simplified set is termed the **canonical cover**
- To define canonical cover we must first define **extraneous attributes**.
 - An attribute of a functional dependency in F is **extraneous** if we can remove it without changing F^+



Extraneous Attributes

- Removing an attribute from the left side of a functional dependency could make it a stronger constraint.
 - For example, if we have $AB \rightarrow C$ and remove B , we get the possibly stronger result $A \rightarrow C$. It may be stronger because $A \rightarrow C$ logically implies $AB \rightarrow C$, but $AB \rightarrow C$ does not, on its own, logically imply $A \rightarrow C$
- But, depending on what our set F of functional dependencies happens to be, we may be able to remove B from $AB \rightarrow C$ safely (that is, $A \rightarrow C$ can be inferred from the original F).
 - For example, suppose that
 - $F = \{AB \rightarrow C, A \rightarrow D, D \rightarrow C\}$
 - Then we can show that F logically implies $A \rightarrow C$ (i.e., $A \rightarrow D, D \rightarrow C \Rightarrow A \rightarrow C$), making B extraneous in $AB \rightarrow C$.
 - That is, removing B from $AB \rightarrow C$ will not change F^+ .



Extraneous Attributes (Cont.)

- Removing an attribute from the right side of a functional dependency could make it a weaker constraint.
 - For example, if we have $AB \rightarrow CD$ and remove C , we get the possibly weaker result $AB \rightarrow D$. It is weaker because using just $AB \rightarrow D$, we can no longer infer $AB \rightarrow C$.
- But, depending on what our set F of functional dependencies happens to be, we may be able to remove C from $AB \rightarrow CD$ safely .
 - For example, suppose that
$$F = \{ AB \rightarrow CD, A \rightarrow C \}.$$
 - Then we can show that even after replacing $AB \rightarrow CD$ by $AB \rightarrow D$, we can still infer $AB \rightarrow C$ (i.e., $AB \rightarrow A, A \rightarrow C \Rightarrow AB \rightarrow C$) and thus $AB \rightarrow CD$, making C extraneous in $AB \rightarrow CD$.



Extraneous Attributes

- An attribute of a functional dependency in F is **extraneous** if we can remove it without changing F^+
- Consider a set F of functional dependencies and the functional dependency $\alpha \rightarrow \beta$ in F .
 - **Remove from the left side:** Attribute A is **extraneous** in α if
 - $A \in \alpha$ and
 - F logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.
 - **Remove from the right side:** Attribute A is **extraneous** in β if
 - $A \in \beta$ and
 - The set of functional dependencies $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ logically implies F .
- **Note:** implication in the opposite direction is trivial in each of the cases above, since a “stronger” functional dependency always implies a weaker one



Testing if an Attribute is Extraneous

- Let R be a relation schema and let F be a set of functional dependencies that hold on R . Consider an attribute in the functional dependency $\alpha \rightarrow \beta$.
- To test if attribute $A \in \alpha$ is extraneous in α
 - Let $\gamma = \alpha - \{A\}$. Check if $\gamma \rightarrow \beta$ can be inferred from F .
 - Compute γ^+ using the dependencies in F
 - If γ^+ includes all attributes in β then, A is extraneous in α
- To test if attribute $A \in \beta$ is extraneous in β
 - Consider the set:
$$F' = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\},$$
 - Compute α^+ using the dependencies in F' check that α^+ contains A ; if it does, A is extraneous in β



Examples of Extraneous Attributes

- Let $F = \{AB \rightarrow CD, A \rightarrow E, E \rightarrow C\}$
- To check if B is extraneous in $AB \rightarrow CD$, we:
 - Compute the attribute closure of A under $F = \{AB \rightarrow CD, A \rightarrow E, E \rightarrow C\}$
 - A^+ is AEC , which does not include CD
 - This implies that B is *not* extraneous
- To check if C is extraneous in $AB \rightarrow CD$, we:
 - Compute $\{AB\}^+$ (the attribute closure of AB) under $F' = \{AB \rightarrow D, A \rightarrow E, E \rightarrow C\}$
 - The closure is $ABCDE$, which includes C
 - This implies that C is extraneous



Canonical Cover

A **canonical cover** for F is a set of dependencies F_c such that

- F logically implies all dependencies in F_c , and
- F_c logically implies all dependencies in F , and
- No functional dependency in F_c contains an extraneous attribute, and
- Each left side of functional dependency in F_c is unique. That is, there do not exist two dependencies $\alpha_1 \rightarrow \beta_1$ and $\alpha_2 \rightarrow \beta_2$ in F_c such that:
 - $\alpha_1 = \alpha_2$



Canonical Cover

- To compute a canonical cover for F :

repeat

Use the union rule to replace any dependencies in F of the form

$$\alpha_1 \rightarrow \beta_1 \text{ and } \alpha_1 \rightarrow \beta_2 \text{ with } \alpha_1 \rightarrow \beta_1 \beta_2$$

Find a functional dependency $\alpha \rightarrow \beta$ in F_c with an extraneous attribute either in α or in β

/* Note: test for extraneous attributes done using F_c , not F */

If an extraneous attribute is found, delete it from $\alpha \rightarrow \beta$

until F_c does not change

- Note: Union rule may become applicable after some extraneous attributes have been deleted, so it has to be re-applied



Example: Computing a Canonical Cover

- $R = (A, B, C)$
 $F = \{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$
- Combine $A \rightarrow BC$ and $A \rightarrow B$ into $A \rightarrow BC$
 - Set is now $\{A \rightarrow BC, B \rightarrow C, AB \rightarrow C\}$
- A is extraneous in $AB \rightarrow C$
 - Check if the result of deleting A from $AB \rightarrow C$ is implied by the other dependencies
 - Yes: in fact, $B \rightarrow C$ is already present!
 - Set is now $\{A \rightarrow BC, B \rightarrow C\}$
- C is extraneous in $A \rightarrow BC$
 - $F' = \{A \rightarrow B, B \rightarrow C, AB \rightarrow C\}$
 - Check if $A \rightarrow C$ is logically implied by F'
 - Yes: using transitivity on $A \rightarrow B$ and $B \rightarrow C$.
 - Can use attribute closure of A in more complex cases
- The canonical cover is:
 $A \rightarrow B$
 $B \rightarrow C$



Lossless Decomposition

- We can use functional dependencies to show when certain decomposition are lossless.
- For the case of $R = (R_1, R_2)$, we require that for all possible relations r on schema R

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

- A decomposition of R into R_1 and R_2 is lossless decomposition if at least one of the following dependencies is in F^+ :
 - $R_1 \cap R_2 \rightarrow R_1$
 - $R_1 \cap R_2 \rightarrow R_2$



Example

- $R = (A, B, C)$
 $F = \{A \rightarrow B, B \rightarrow C\}$

- $R_1 = (A, B), R_2 = (B, C)$

- Lossless decomposition:

$$R_1 \cap R_2 = \{B\} \text{ and } B \rightarrow BC$$

- $R_1 = (A, B), R_2 = (A, C)$

- Lossless decomposition:

$$R_1 \cap R_2 = \{A\} \text{ and } A \rightarrow AB$$



Dependency Preservation

- Testing functional dependency constraints each time the database is updated can be costly
- It is useful to design the database in a way that constraints can be tested efficiently.
- If testing a functional dependency can be done by considering just one relation, then the cost of testing this constraint is low
- When decomposing a relation it may be no longer possible to do the testing without having to perform a Cartesian Product.
- A decomposition that makes it computationally hard to enforce functional dependency is said to be NOT **dependency preserving**.



Dependency Preservation Example

- Consider a schema:

dept_advisor(s_ID, i_ID, department_name)

- With function dependencies:

$i_ID \rightarrow dept_name$

$s_ID, dept_name \rightarrow i_ID$

- In the above design we are forced to repeat the department name once for each time an instructor participates in a *dept_advisor* relationship.
- To fix this, we need to decompose *dept_advisor*
- Any decomposition will not include all the attributes in
 $s_ID, dept_name \rightarrow i_ID$
- Thus, the composition NOT be dependency preserving



Dependency Preservation

- Let F_i be the set of dependencies F^+ that include only attributes in R_i .
 - A decomposition is **dependency preserving**, if
$$(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$$
- Using the above definition, testing for dependency preservation take exponential time.
- Note that if a decomposition is NOT dependency preserving then checking updates for violation of functional dependencies may require computing joins, which is expensive.



Dependency Preservation (Cont.)

- Let F be the set of dependencies on schema R and let $R_1, R_2, \dots, R_n$ be a decomposition of R .
- The **restriction** of F to R_i is the set F_i of all functional dependencies in F^+ that include **only** the attributes of R_i .
- Since all functional dependencies in a restriction involve attributes of only one relation schema, it is possible to test such a dependency for satisfaction by checking only one relation.
- Note that the definition of restriction uses all dependencies in F^+ , not just those in F .
- The set of restrictions $F_1, F_2, \dots, F_n$ is the set of functional dependencies that can be checked efficiently.



Testing for Dependency Preservation

- To check if a dependency $\alpha \rightarrow \beta$ is preserved in a decomposition of R into $R_1, R_2, \dots, R_n$, we apply the following test (with attribute closure done with respect to F)
 - $result = \alpha$
repeat
 for each R_i in the decomposition
 $t = (result \cap R_i)^+ \cap R_i$
 $result = result \cup t$
 until ($result$ does not change)
 - If $result$ contains all attributes in β , then the functional dependency $\alpha \rightarrow \beta$ is preserved.
- We apply the test on all dependencies in F to check if a decomposition is dependency preserving
- This procedure takes polynomial time, instead of the exponential time required to compute F^+ and $(F_1 \cup F_2 \cup \dots \cup F_n)^+$

$R = (A, B, C)$

$F = \{A \rightarrow B, B \rightarrow C\}$

Decomposition $R_1 = (A, B),$
 $R_2 = (B, C)$

For $A \rightarrow B$

$Result = \{A\}$

For $R_1,$

$t = A^+ \cap \{AB\} = \{AB\}$

$result = \{AB\}$

For R_2

$t = (\{AB\} \cap \{BC\})^+ \cap \{BC\}$

$= B^+ \cap \{BC\} = \{BC\}$

$result = \{ABC\}$

Thus $A \rightarrow B$ is preserved.



Example

- $R = (A, B, C)$
 $F = \{A \rightarrow B$
 $\quad B \rightarrow C\}$
Key = $\{A\}$
- R is not in BCNF
- Decomposition $R_1 = (A, B), R_2 = (B, C)$
 - R_1 and R_2 in BCNF
 - Lossless-join decomposition
 - Dependency preserving

$$F_1 = \{A \rightarrow B\}, F_2 = \{B \rightarrow C\}$$

$$F_1 \cup F_2 = F$$



Algorithm for Decomposition Using Functional Dependencies



Testing for BCNF

- To check if a non-trivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF
 1. compute α^+ (the attribute closure of α), and
 2. verify that it includes all attributes of R , that is, it is a superkey of R .
- **Simplified test:** To check if a relation schema R is in BCNF, it suffices to check only the dependencies in the given set F for violation of BCNF, rather than checking all dependencies in F^+ .
 - If none of the dependencies in F causes a violation of BCNF, then none of the dependencies in F^+ will cause a violation of BCNF either.
- However, the **simplified test** using only F is incorrect when testing a relation in a decomposition of R
 - Consider $R = (A, B, C, D, E)$, with $F = \{A \rightarrow B, BC \rightarrow D\}$
 - Decompose R into $R_1 = (A, B)$ and $R_2 = (A, C, D, E)$
 - Neither of the dependencies in F contain only attributes from (A, C, D, E) so we might be misled into thinking R_2 satisfies BCNF. (i.e., $ACDE$ as candidate key, so that every attribute depends only on the candidate key)
 - In fact, dependency $AC \rightarrow D$ in F^+ shows R_2 is not in BCNF.



Testing Decomposition for BCNF

To check if a relation R_i in a decomposition of R is in BCNF

- Either test R_i for BCNF with respect to the **restriction** of F^+ to R_i (that is, all FDs in F^+ that contain only attributes from R_i)
- Or use the original set of dependencies F that hold on R , but with the following test:
 - for every set of attributes $\alpha \subseteq R_i$, check that α^+ (the attribute closure of α) either includes no attribute of $R_i - \alpha$, or includes all attributes of R_i .
- If the condition is violated by some $\alpha \rightarrow \beta$ in F^+ , the dependency
$$\alpha \rightarrow (\alpha^+ - \alpha) \cap R_i$$
can be shown to hold on R_i , and R_i violates BCNF.
- We use above dependency to decompose R_i



BCNF Decomposition Algorithm

```
result := {R };
done := false;
compute  $F^+$ ;
while (not done) do
    if (there is a schema  $R_i$  in result that is not in BCNF)
        then begin
            let  $\alpha \rightarrow \beta$  be a nontrivial functional dependency that
                holds on  $R_i$  such that  $\alpha \rightarrow R_i$  is not in  $F^+$ ,
                and  $\alpha \cap \beta = \emptyset$ ;
            result := (result –  $R_i$ )  $\cup$  ( $R_i - \beta$ )  $\cup$  ( $\alpha, \beta$ );
        end
    else done := true;
```

Note: each R_i is in BCNF, and decomposition is lossless-join.



Example of BCNF Decomposition

- *class* (*course_id*, *title*, *dept_name*, *credits*, *sec_id*, *semester*, *year*, *building*, *room_number*, *capacity*, *time_slot_id*)
- Functional dependencies:
 - *course_id* → *title*, *dept_name*, *credits*
 - *building*, *room_number* → *capacity*
 - *course_id*, *sec_id*, *semester*, *year* → *building*, *room_number*, *time_slot_id*
- A candidate key {*course_id*, *sec_id*, *semester*, *year*}.
- BCNF Decomposition:
 - *course_id* → *title*, *dept_name*, *credits* holds $\alpha \rightarrow \beta$
 - but *course_id* is not a superkey.
 - We replace *class* by:
 - *course*(*course_id*, *title*, *dept_name*, *credits*) $(R - \beta) \cup (\alpha, \beta);$
 - *class-1* (*course_id*, *sec_id*, *semester*, *year*, *building*, *room_number*, *capacity*, *time_slot_id*)



BCNF Decomposition (Cont.)

- *course* is in BCNF
 - How do we know this?
- *building, room_number* → *capacity* holds on *class-1* $\alpha \rightarrow \beta$
 - but $\{building, room_number\}$ is not a superkey for *class-1*.
 - We replace *class-1* by:
 - *classroom* (*building, room_number, capacity*)
 - *section* (*course_id, sec_id, semester, year, building, room_number, time_slot_id*)
- *classroom* and *section* are in BCNF. $(R_i - \beta) \cup (\alpha, \beta);$



Third Normal Form

- There are some situations where
 - BCNF is not dependency preserving, and
 - efficient checking for FD violation on updates is important
- Solution: define a weaker normal form, called Third Normal Form (3NF)
 - Allows some redundancy (with resultant problems; we will see examples later)
 - But functional dependencies can be checked on individual relations without computing a join.
 - There is always a lossless-join, dependency-preserving decomposition into 3NF.



3NF Example -- Relation *dept_advisor*

- *dept_advisor* (*s_ID*, *i_ID*, *dept_name*)
 $F = \{s_ID, dept_name \rightarrow i_ID, i_ID \rightarrow dept_name\}$
- Two candidate keys: $\{s_ID, dept_name\}$, and $\{i_ID, s_ID\}$
- *R* is in 3NF
 - $s_ID, dept_name \rightarrow i_ID \quad s_ID$
 - $s_ID, dept_name$ is a superkey
 - $i_ID \rightarrow dept_name$
 - $dept_name$ is contained in a candidate key



Testing for 3NF

- Need to check only FDs in F , need not check all FDs in F^+ .
- Use attribute closure to check for each dependency $\alpha \rightarrow \beta$, if α is a superkey.
- If α is not a superkey, we have to verify if each attribute in β is contained in a candidate key of R
 - This test is rather more expensive, since it involve finding candidate keys
 - Testing for 3NF has been shown to be NP-hard
 - Interestingly, decomposition into third normal form (described shortly) can be done in polynomial time



3NF Decomposition Algorithm

```
Let  $F_c$  be a canonical cover for  $F$ ;  
 $i := 0$ ;  
for each functional dependency  $\alpha \rightarrow \beta$  in  $F_c$  do  
  if none of the schemas  $R_j$ ,  $1 \leq j \leq i$  contains  $\alpha \beta$   
    then begin  
       $i := i + 1$ ;  
       $R_i := \alpha \beta$   
    end  
if none of the schemas  $R_j$ ,  $1 \leq j \leq i$  contains a candidate key for  $R$   
  then begin  
     $i := i + 1$ ;  
     $R_i :=$  any candidate key for  $R$ ;  
  end  
/* Optionally, remove redundant relations */  
repeat  
if any schema  $R_j$  is contained in another schema  $R_k$   
  then /* delete  $R_j$  */  
     $R_j = R_k$ ;  
     $i = i - 1$ ;  
return  $(R_1, R_2, \dots, R_i)$ 
```



3NF Decomposition Algorithm (Cont.)

Above algorithm ensures

- Each relation schema R_i is in 3NF
- Decomposition is dependency preserving and lossless-join
- Proof of correctness is at end of this presentation ([click here](#))



3NF Decomposition: An Example

- Relation schema:

cust_banker_branch = (customer_id, employee_id, branch_name, type)

- The functional dependencies for this relation schema are:

- *customer_id, employee_id → branch_name, type*
- *employee_id → branch_name*
- *customer_id, branch_name → employee_id*

- We first compute a canonical cover

- *branch_name* is extraneous in the r.h.s. of the 1st dependency
- No other attribute is extraneous, so we get $F_C =$

customer_id, employee_id → type

employee_id → branch_name

customer_id, branch_name → employee_id



3NF Decomposition Example (Cont.)

- The **for** loop generates following 3NF schema:
 - (customer_id, employee_id, type)*
 - (employee_id, branch_name)*
 - (customer_id, branch_name, employee_id)*
 - Observe that *(customer_id, employee_id, type)* contains a candidate key of the original schema, so no further relation schema needs be added
- At end of for loop, detect and delete schemas, such as *(employee_id, branch_name)*, which are subsets of other schemas
 - result will not depend on the order in which FDs are considered
- The resultant simplified 3NF schema is:
 - (customer_id, employee_id, type)*
 - (customer_id, branch_name, employee_id)*



Comparison of BCNF and 3NF

- It is always possible to decompose a relation into a set of relations that are in 3NF such that:
 - The decomposition is lossless
 - The dependencies are preserved
- It is always possible to decompose a relation into a set of relations that are in BCNF such that:
 - The decomposition is lossless
 - It may not be possible to preserve dependencies.



End of Chapter 7